

They stay responsive,
and storage stops
growing.

SURFACE		CLOSED
Many sessions on one machine		0
Slack	no listed issues	0
Cassy Cloud	no listed issues	0
Folders outside the project	no listed issues	0
Checks and sync	no listed issues	0
Release	no listed issues	0
Many-agent scale	no listed issues	0
Factory	no listed issues	0
Memory	no listed issues	0
Violet	no listed issues	0
Write roots (GH #1169)	no listed issues	0
Visual QA	no listed issues	0

The closure map is empty until release evidence names verified issues.

One dot, one closed issue. 0 issue-bearing surfaces.

[Read the map data](#) ↓

Release at a glance

O

Issues closed

Named, verified closed issues in this report

115m 41s

Green to published

gate.green.epoch to the published receipt, when available

O

Features added

Top-level Added entries in the CHANGELOG section

2m 30s — within budget (10m 0s budget)

Tag to published

First tag workflow to publication; measured budget result

O

Changed entries

Top-level Changed entries in the CHANGELOG section

4

Manual interventions

Distinct rescued stages plus recorded hand fixes; receipt INTERVENTIONS

O

Fix entries

Top-level Fixed entries in the CHANGELOG section

▼ Change map · data and reading guide

One filled dot represents one closed issue. Dots have equal weight: this is a count of closures, not a measured impact score.

Change map data · closed issues by surface

SURFACE	ISSUES CLOSED	ISSUE NUMBERS
Many sessions on one machine	0	No listed issues
Slack	0	No listed issues
Cassy Cloud	0	No listed issues
Folders outside the project	0	No listed issues
Checks and sync	0	No listed issues
Release	0	No listed issues
Many-agent scale	0	No listed issues
Factory	0	No listed issues
Memory	0	No listed issues
Violet	0	No listed issues
Write roots (GH #1169)	0	No listed issues
Visual QA	0	No listed issues

Total

0 0 issue-bearing themes

Source: CHANGELOG /home/pippenz/Petrastella/cassy/.cas/release-v3.50.0/CHANGELOG.md, release-notes /home/pippenz/Petrastella/cassy/.cas/release-v3.50.0/docs/release-notes/2026-10-10-v3.50.0-slack.md, GitHub Richards-LLC/cassy release v3.50.0; retrieved 2026-10-11T02:43:26Z.

What you can do now

02 / 06

Many sessions on one machine

No more freezes

WAS with a dozen Cassy sessions running, the screen and messages could freeze for seconds while one session waited on another. →

NOW the main screen never waits on the database, so it keeps up. In a test with 16 sessions working at once, the main screen finished 99.5% of its updates in under a tenth of a second, where before it stalled for more than two seconds at a time.

Storage stays bounded

WAS Cassy's database grew without limit, mostly from repeated delivery notes and old message transcripts. →

NOW routine logs are cleared after 14 days and transcripts after 7, while task and code history is kept.

One code index per project

WAS every copy of a project built its own code index, and copies left behind kept growing. →

NOW there is one index per project, leftovers are cleaned up, and `cas doctor --fix` removes stray copies.

Codex starts one Cassy, not two

WAS each Codex session started two Cassy servers that competed for the same files. →

NOW it starts one.

The screen no longer pauses to refresh

WAS every two seconds the main screen stopped to reread every task, agent and recent event, so with many sessions it hitched for a few tenths of a second each time. →

NOW that reading happens in the background and the screen picks up the result a moment later, so it never stops to wait.

No more rare freezes when work finishes

WAS very occasionally a Cassy process could freeze for good when two of its database connections closed at the same moment. →

NOW that can't happen any more.

Slack

Hand-offs stay with people

WAS a Slack message that tagged a colleague woke Cassy as if it were meant for Violet. →

NOW Cassy sees it is for that person and leaves it to them.

See what Slack watching is doing

WAS there was no way to tell which channels Cassy was watching or whether its Slack link worked. →

NOW the status screen lists each watched channel, and `cas doctor` warns when the link keeps failing.

Connect a Slack channel yourself

WAS linking a Slack channel to a project needed a manual request. →

NOW `cas integrate violet --channel <name>` links it.

Violet setup no longer loops

WAS setup could keep saying "run `cas login`" for a Slack key that signing in never creates. →

NOW it uses this machine's own key and names exactly what to fix.

Claude Code check is honest

WAS setup said Claude Code was "already current" while Slack rejected its key. →

NOW it checks with Slack, says when the key is refused, and repairs other Claude Code profiles on the machine too.

Connected tools come back on their own

WAS after one dropped connection, Slack and other connected tools stayed unreachable until Cassy restarted, and a stuck request could hang for over 15 minutes. →

NOW Cassy reconnects on the next request, gives up on a stuck one after 90 seconds, and says plainly what is wrong.

Cassy Cloud

Answers to terminal questions on your phone

WAS Cassy Cloud showed your terminal questions but not the answers. →

NOW each answer appears under its question.

Switching conversations always works

WAS tapping a conversation sometimes did nothing or opened the wrong one, because the list reordered or refreshed while your finger was on it. →

NOW the conversation you pressed opens every time, and an older load never replaces a newer choice.

Install Cassy Cloud as an app

WAS Cassy Cloud's icon was a pale, flat mark that was hard to see, and the site could not be installed as an app. →

NOW a new violet icon, and Cassy Cloud installs as an app on a phone or computer, fits around the iPhone notch, with a sharp icon on light and dark home screens.

Folders outside the project

Let a project write to other folders

WAS Cassy's agents could only write inside their own project, so updating a config folder elsewhere meant copying files by hand. →

NOW you can let a project write to folders outside its own, from the terminal or from Commander on your phone, for the whole project or just one task.

You stay in charge

WAS nothing said which outside folders an agent could touch. →

NOW only you can add a folder, after confirming it; agents can't add one themselves, each write there is logged, and with no folders added nothing changes.

Checks and sync

Screenshot checks wait for the real page

WAS strict screenshot checks passed on a page still showing a loading spinner, and flagged text that was only fading in. →

NOW they wait for the page to render and settle, and still fail on text that stays invisible.

Tasks owned by another project

WAS Cloud sync called a task already owned by another project an "unrecognized" problem. →

NOW it names that project and the fix.

Long jobs are not "stalled"

WAS a session busy in one long job set off repeated "stalled" alerts. →

NOW it is recognised as busy, and alerts are combined.

Many-agent scale

Lock-free daemon loop

WAS the factory loop ran store reads and writes inline and could wait seconds on SQLite busy or flocks. →

NOW each pass has a 50 ms per-thread wait budget, CI-watch, lifecycle outbox, attention relays and task-dialog loads run on an ordered store worker, and `loop.json` reports max, p99 and ≥ 100 ms passes per window. Measured with `scripts/many-agent-load.py` (16 `cas serve` agents plus a headless daemon, code index on, two synthetic writers each holding the write lock about 400 ms in every second, 300 s): 3 of 636 daemon passes took 100 ms or more (0.47%, max 121 ms), MCP task/coordination p99 1.74 s and no wait on the task-sync lease. Without the writers it was 0.15% (max 120 ms) and p99 0.19 s, and at 32 agents 0.44% (max 114 ms) and p99 1.94 s. On 3.49.0 (0d670c33c), 16 agents under the same pressure (code index off) gave p99 2.05 s, 41 lease waits of 0.4 s or more and daemon passes of at least 2.35 s.

Task-store opens

WAS every open reconciled under the exclusive `task-sync-intents.lock`. →

NOW the lock is a shared per-process lease, mutations serialize on 64 per-entity stripes, reconcile takes a 100 ms bounded `BEGIN IMMEDIATE` and defers when busy, and opens reconcile once per process then every 60 s.

Pooled writers

WAS a pooled caller held the process-wide connection mutex through six busy waits (~31.6 s). →

NOW `begin_immediate_pooled` takes the mutex per attempt and drops it before each backoff; `history commit_batch` takes the write lock first and skips unchanged FTS rows.

One code index per project

WAS every worker and worktree indexed into the shared store, leaving per-worktree copies. →

NOW only the canonical checkout's process indexes, reconciles and purges; worktree copies are purged in bounded rowid batches with FK enforcement suspended per transaction; doctor reports per-repo counts and `--fix` purges.

One `cas serve` per Codex worker

WAS a user-level `[mcp_servers.cas]` plus the spawn-injected `cs` key started two serves per worker. →

NOW other Cassy keys are disabled with a complete inline table and the config writer converges on `cs`.

Bounded event log

WAS events was never pruned (1.08 M rows, ~450 MB with indexes, 800 k of them `supervisor_injected`). →

NOW telemetry types expire after `factory.event_telemetry_retention_days` (14) in $\leq 1,000$ -row `IMMEDIATE` batches on the canonical daemon every 15 min, ungated on idle; lifecycle types are kept, identical injection retries are recorded once, and no-op re-offers are recorded as reoffered.

Prompt and queue retention

WAS `prompts`, `prompt_queue` and `supervisor_queue` grew unbounded. →

NOW `messages_json` is nulled after `factory.prompt_transcript_retention_days` (7), delivered queue and outbox rows are removed after `factory.supervisor_queue_retention_days` (14), all in the same batched driver.

Refresh reads off the loop

WAS the 2 s director refresh read every task, agent and recent event on the loop thread and delivery reread the snapshot before revalidating, while the agent and event stores took the process-wide connection with a plain mutex outside the pass budget (13.9% of passes ≥ 100 ms at $N=16$, max 402 ms). →

NOW both reads run on one background reader with its own WAL connection, the loop applies the finished snapshot on a later pass (panels stale by at most one refresh; events are revalidated against a read taken after detection), the agent and event stores honour the 50 ms pass budget, and `loop.json` adds `phase_latency` (entries, max and ≥ 100 ms runs per phase, with refresh split into its steps).

SQLite close deadlock

WAS bundled SQLite 3.51.1 could deadlock forever when two WAL connections in one process closed at once (`unixLock` held the inode lock mutex while `unixIsSharingShmNode` took the global VFS mutex, and `unixClose` took them in the opposite order); no busy timeout or wait budget bounds that wait. →

NOW `rusqlite` 0.39 bundles SQLite 3.51.3, which carries the 3.51.2 fix for that deadlock and the WAL-reset corruption fix, and a test fails if an older SQLite is bundled.

Violet

Same-hub project override

WAS a project `[servers.violet]` at the machine's hub URL naming another machine's token kept integrate stale and advised `cas login` (GH #1164). →

NOW it is dropped, the machine token verifies the hub, a 401 names the variable and the re-mint path, and no hub-token advice names `cas login`.

Claude Code verification

WAS `claude-code` reported "already current" structurally and only the `CLAUDE_CONFIG_DIR` profile was reconciled. →

NOW each entry carries the authenticated `tools/list` verdict, and every `~/ .claude*` profile that already registers violet is rewritten from literal or legacy refs to env references.

Push-wake follow-ups

WAS a wake could not tell a hand-off from a mention of Violet, and watch state was invisible. →

NOW `message.mentions` with only people marks the wake `addressed="human"`, `WatchBook.relay` records every claim tick, `cas factory` status and `--json` show watches and relay health, and doctor adds a `violet wake` row.

--channel mapping

WAS `channel`→project mappings were registered by hand. →

NOW `cas` integrate violet `--channel` calls hub PUT `/api/channels/{channel}` with the Cloud login, with `--channel-replace` and `--channel-remove`.

MCP upstream reconnect **WAS** after a transport error the proxy dropped the upstream, so `mcp_execute` said "absent ... restore its credential" for the rest of the session while `proxy_health` stayed healthy, and a hung call held ~1000 s (GH #1168). →

NOW a dropped upstream reconnects on demand once its backoff is due, every call is bounded client-side (90 s, `call_timeout_secs` in `proxy.toml`), `proxy_health` is republished when a call changes health, and the absent message names the cause.

Cassy Cloud

Terminal answers mirrored **WAS** the Commander reply mirror ran only after a verified paired-device send, so a terminal-only operator saw questions without answers. →

NOW any operator activity activates it, a ±120 s terminal-question match threads the answer via `reply_to`, and harness prompts are excluded.

Conversation switching **WAS** a catalog update between press and release lost the tap: grouped rows re-sorted and `insertBefore` moved another row under the finger, and a session start or end rebuilt the shell and detached the pressed button. →

NOW `ConversationList` holds the pressed row by key for the gesture, redirects a stray click to it, opens it on a timer when no click arrives (0 ms mouse, 300 ms touch, late touch click absorbed), and a stale open never attaches after a newer selection.

App icon and install **WAS** the mark was flat ink inheriting `--color-action` (pale #A9B3FF on the dark Glass field) and there was no web app manifest. →

NOW a `--look-send` violet-gradient tile (#7656FA→#5B3FE0→#3E5BEA) with white and aurora ribbons; `render-brand-icons.mjs` paints favicon 16/32, apple-touch 180 and 192/512 any and maskable icons from `favicon.svg`; `manifest.webmanifest` (Cassy, standalone, theme #5B3FE0, background #0E0C20) and the iOS meta make it installable; headers and sheets pad `env(safe-area-inset-*)` (e2e/`installed-safe-area.spec.ts`); `DESIGN.md` "Brand" records sizes and contrast.

Write roots (GH #1169)

Operator write roots **WAS** the workspace contract sanctioned only the worktree, artifacts, scratch and the harness scratchpad, with no override. →

NOW `.cas/operator/write-policy.toml` adds project roots (`cas config set factory.write_roots "~/a,~/b:create+edit+delete"`) and per-task grants (`cas config grant-write`, expiring at task close), each judged per create/edit/delete mode, audited as `workspace_write_root_used`, and listed in refusals.

Operator-only, guardrail-grade **WAS** there was no operator-only setting; anything an agent could run, it could configure. →

NOW `Config::set` refuses the `factory.write_roots` key specifically; the CLI refuses agent env markers, agent or `cas serve` ancestors, `factory` cgroups and non-TTY runs, and asks for a typed confirmation; `PreToolUse` refuses agent writes under `.cas/operator/` and agent calls of the commands. Agents share the operator's Unix user, so the hook is the gate, not a security boundary.

Commander grants	WAS grants needed the operator's terminal. → NOW <code>POST /v1/sessions/{s}/write-grants</code> (DPoP device auth plus <code>factory:manage</code>) has the hub write or revoke the grant as <code>commander-device:<id></code>, note the task, audit it and post a verified operator receipt; hub-web adds a Write access panel with confirmation.
Unchanged by default	WAS <code>sed -i</code> and <code>mv</code> sources were never judged. → NOW with a policy file they are judged as edits; without one the contract is exactly as before.

Visual QA

Readiness	WAS <code>--strict</code> passed on a spinner-only page (GH #1159). → NOW a bounded readiness wait (<code>--ready-selector</code>, <code>--ready-timeout-ms</code>) and a non-allowlistable <code>page-not-ready</code> finding.
Settle on quiescence	WAS <code>settlePage</code> finished only animations already running, so a post-load enter transition read as <code>invisible-text</code> (GH #1158). → NOW <code>settle</code> waits for 500 ms without DOM mutations or animations, bounded at 3 s.
Scoped comparisons	WAS a per-source report with <code>renders[].issues</code> could not be compared (GH #1166). → NOW it is flattened and paired by the same rules.

Closes, delivery and workers

Reviewed drop	WAS <code>reviewed-drop</code> rejected superseding commits older than the delivery anchor (GH #1160). → NOW they are accepted with per-path coverage on the target.
No-code close	WAS a no-code task without a work target needed a branch (GH #1167). → NOW only commits naming the task count as its own; retained anchors still bind.
Delivery stalls	WAS a Codex worker busy in one long turn raised repeated <code>worker_delivery_stalled</code> alerts (GH #1163). → NOW executing liveness defers the probe, recipient replies and notes retire it, and alerts coalesce per worker per 5-minute window.
Build guard	WAS <code>env -u VAR cargo nextest ...</code> named <code>VAR</code> as the executable and ran raw cargo. → NOW <code>env</code> and <code>sudo</code> value options consume their value and <code>env -s</code> payloads are inspected.
Stale index.lock	WAS a killed <code>git status</code> probe could leave a zero-byte <code>index.lock</code>. → NOW read-only probes run with <code>GIT_OPTIONAL_LOCKS=0</code>, and a worker may remove an unheld stale lock in its own <code>gitdir</code>.

Cloud duplicates	WAS <code>duplicate_of_other_project</code> was reported as an unrecognized reason. → NOW the owning project is kept in diagnostics, with the <code>alias-fold</code> and <code>adopt-aliases</code> remedy.
Release train	
Integration identity	WAS the integration branch name was derived from the checkout directory, and <code>epics</code> merged to <code>main</code> by PR had no assembly path. → NOW it is persisted as <code>cas.integrationBranch</code>, from-main assembly is supported, and its gate refuses <code>--reuse</code> and <code>--only</code>.
Gate scratch and renderer	WAS gate scratch could land on another filesystem, and a missing report renderer surfaced only after publish. → NOW scratch defaults beside the checkout, and preflight launches the renderer's headless Chromium.

Fixes ledger

04 / 06

Issue register · 0 verified closures

ISSUE	STATE	CLOSED	THEME
-------	-------	--------	-------

Install

05 / 06



```
cas update # install v3.50.0 through the project's normal release channel
```

Change map data · closed issues by surface

ASSET	DIGEST	SIZE
cas-aarch64-apple-darwin.tar.gz	sha256:ac1dd75c41a441e45866cd107c79701642feef5fbd5466697c4865e427a0cd1c	31105369
cas-x86_64-unknown-linux-gnu.tar.gz	sha256:ca8183207ede54771e9899a9e6b7dfb61cc01006e1a2a9f0e07a8259c387c83f	36205873

Evidence and scope

06 / 06

Retrieved: 2026-10-11T02:43:26Z.

Source: CHANGELOG /home/pippenz/Petrastella/cassy/.cas/release-v3.50.0/CHANGELOG.md, release-notes /home/pippenz/Petrastella/cassy/.cas/release-v3.50.0/docs/release-notes/2026-10-10-v3.50.0-slack.md, GitHub Richards-LLC/cassy release v3.50.0; retrieved 2026-10-11T02:43:26Z.

GitHub issue theme assignment uses labels first, then the keyword table in the CLI. Any issue classified as Unclassified is shown in the map so a human can correct the Markdown before rendering.

Release-notes draft: docs/release-notes/2026-10-10-v3.50.0-slack.md.

CHANGELOG source: CHANGELOG.md.

GitHub release: <<https://github.com/Richards-LLC/cassy/releases/tag/v3.50.0>> (published or unknown).

Release receipt: /home/pippenz/.cas/artifacts/release/v3.50.0-release-v3.50.0/release-published.receipt.

Source warnings:

- GitHub issues: referenced closures unavailable or not closed: #1158, #1159, #1160, #1163, #1164, #1165, #1166, #1167, #1168, #1169

cas-src v3.50.0 / Release report

[Markdown source](#) · [Print edition](#)